

gcp-deploy

Online Boutique on Google Cloud — architecture, decisions and runbook

REGION

europe-west4

CLUSTER

GKE Autopilot, regional, private nodes

STATE

GCS, versioned and locked

WORKLOAD

11 microservices + redis-cart, v0.10.6

CONTENTS**Context, requirements and scope**

- The brief
- Mandatory requirements
- Non-goals
- Assumptions
- Constraints
- What "done" looks like

Architecture

- Level 1 — system context
- Level 2 — infrastructure containers
- Level 3 — application components
- Address plan
- Repository layout
- Failure modes

Security posture

- Threat model
- Identity: two planes
- Network posture
- Workload hardening
- Supply chain
- Known gaps

Runbook

- The stages
- Prerequisites
- Deploy
- Why the stages are cut where they are
- Proving it works
- Troubleshooting

- Teardown

Cost model

- What Autopilot actually charges for
- Estimated monthly bill
- Cost controls already in the repository
- Free tier now, premium in production
- Trial-account fine print

Roadmap

- Next, in order
- Deliberately not on this list
- How the documentation scales

Architecture decision records**ADR 0001 — GKE Autopilot as the compute platform****ADR 0002 — Private nodes with a guarded public control-plane endpoint****ADR 0003 — A dedicated node service account, not the Compute default****ADR 0004 — Versioned GCS remote state with native locking****ADR 0005 — Implement the mandated CIDR sizing, and record the disagreement****ADR 0006 — Binary Authorization wired but disabled by default****ADR 0007 — Pin upstream manifests to a release tag, not a branch****ADR 0008 — An explicit saved-plan gate before apply**

Context, requirements and scope

What was asked for, what was deliberately left out, and what the rest of these documents assume. Read this first — every later decision is justified against the constraints recorded here.

The brief

Stand up an empty Google Cloud project into a private, least-privilege Kubernetes environment, entirely through infrastructure as code, and deploy Google's [Online Boutique](#) demo application onto it through something shaped like a delivery pipeline. The work is reviewed by a panel, so the reasoning behind each choice is part of the deliverable rather than an appendix to it.

Mandatory requirements

Each maps to a specific resource, so compliance is verifiable in code rather than asserted in prose.

#	REQUIREMENT	WHERE IT LIVES	HOW
R1	Nodes on <code>10.0.0.0/16</code>	<code>modules/network/main.tf</code>	Primary range of the GKE subnet
R2	Pods on <code>10.1.0.0/16</code>	<code>modules/network</code> → <code>modules/gke</code>	Secondary range <code> pods</code> , bound via <code>ip_allocation_policy</code>
R3	Services on <code>10.2.0.0/16</code>	same	Secondary range <code> services</code> , bound via <code>ip_allocation_policy</code>
R4	Least privilege throughout	<code>modules/gke/main.tf</code>	Dedicated node service account replacing the Compute default; private nodes; NAT-only egress; authorized networks; Shielded nodes and no privileged Pods enforced by Autopilot
R5	Engineers collaborate without side effects	<code>bootstrap/</code> , <code>versions.tf</code> , <code>.gitignore</code>	Versioned remote state with native locking; pinned providers; modules as ownership boundaries; APIs enabled in code
R6	Application deployed via a pipeline	<code>scripts/50-deploy.sh</code>	Mock CD job: fail-fast, idempotent, health-gated, smoke-tested, pinned to a release tag. One of seven ordered stages any orchestrator can drive

Non-goals

Stated explicitly, because the fastest way to misread this repository is to measure it against a target it was never aiming at.

- **Multi-region or multi-cluster.** One regional cluster in `europe-west4` . Regional gives zone-level resilience; anything beyond that is a cost and complexity decision nobody has made yet.
- **Multi-tenancy.** Per-tenant namespaces are the direction of travel, but nothing here implements tenant isolation, quotas or per-tenant policy.
- **Production-grade delivery.** `50-deploy.sh` is deliberately a stand-in for the final sync step of a real pipeline. There is no build, no test stage, no image signing, no GitOps

controller. See [ADR 0007](#) and [06-roadmap](#).

- **Modifying the application.** The upstream manifests are applied as published. No forks, no patches, no re-images — so the repository is about the platform, not the app.
- **Persistent application data.** `redis-cart` uses an `emptyDir`. Carts do not survive a Pod restart, and that is upstream's design, not an oversight here.
- **Cost optimisation beyond the structural.** Autopilot's per-Pod billing and a zero-replica load generator are in scope; committed-use discounts and workload right-sizing are not, because there is no steady-state load profile to right-size against.
- **A service mesh.** Online Boutique speaks gRPC service-to-service without one. Istio or Cloud Service Mesh would add mTLS and traffic policy, and would also add a control plane nobody asked for.

Assumptions

Challenge these first — if one is wrong, the conclusion downstream of it is probably wrong too.

1. **One project, one environment.** Environment separation would be a `envs/` layer over the same modules, not a change to them.
2. **The reviewer can reach the control plane from an arbitrary address.** This is what justifies `0.0.0.0/0` in the authorized networks, and it is the assumption most worth revisiting first. See [ADR 0002](#).
3. **Upstream images are unsigned and stay that way.** Binary Authorization is therefore a demonstrated capability rather than an enforced control. See [ADR 0006](#).
4. **The account is a free trial or credit-backed.** This shapes teardown discipline and quota expectations, not the architecture. See [05-cost](#).
5. **No compliance regime applies.** No data residency, audit retention or sovereignty requirement has been stated. Several choices — log retention in particular — would change if one had been.

Constraints

CONSTRAINT	CONSEQUENCE
Mandated /16 ranges	Wildly oversized for the workload, and implemented as specified. The disagreement is recorded in ADR 0005 rather than silently corrected.
Free-trial quotas	Regional CPU and in-use IP address quotas are lower than on a paid account; an <code>apply</code> that fails usually fails on one of these.
Unsigned upstream images	Admission control can be wired but not enforced.
Reviewed by reading, not just running	Structure, comments and decision records carry as much weight as the resources they create.

What "done" looks like

The environment is correct when all five hold, each checkable by a command in [04-runbook](#):

1. The three mandated ranges exist and are *bound to the cluster*, not merely present in the VPC.

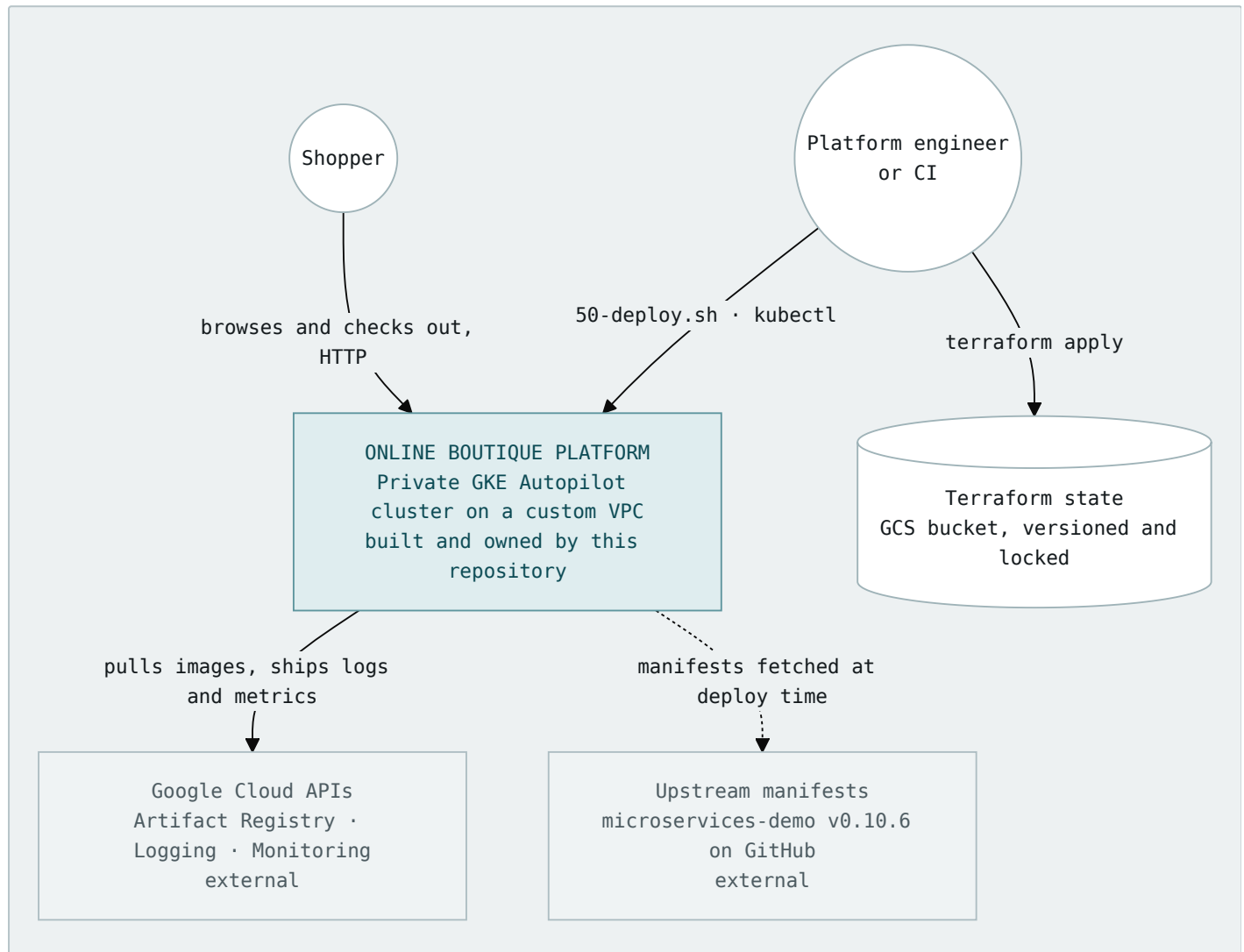
2. No node has an external IP, and egress leaves through Cloud NAT.
3. Nodes run as a dedicated service account, not the Compute Engine default.
4. A second concurrent `terraform apply` blocks rather than corrupting state.
5. The frontend answers HTTP 200 on a load balancer address, with every Deployment reporting `Available`.

Architecture

Three views at decreasing altitude — system context, infrastructure containers, application components — then the address plan, the code that builds it, and how it fails. Decisions are recorded separately in `decisions/`; this document describes *what exists*, and links to *why* rather than restating it.

Level 1 – system context

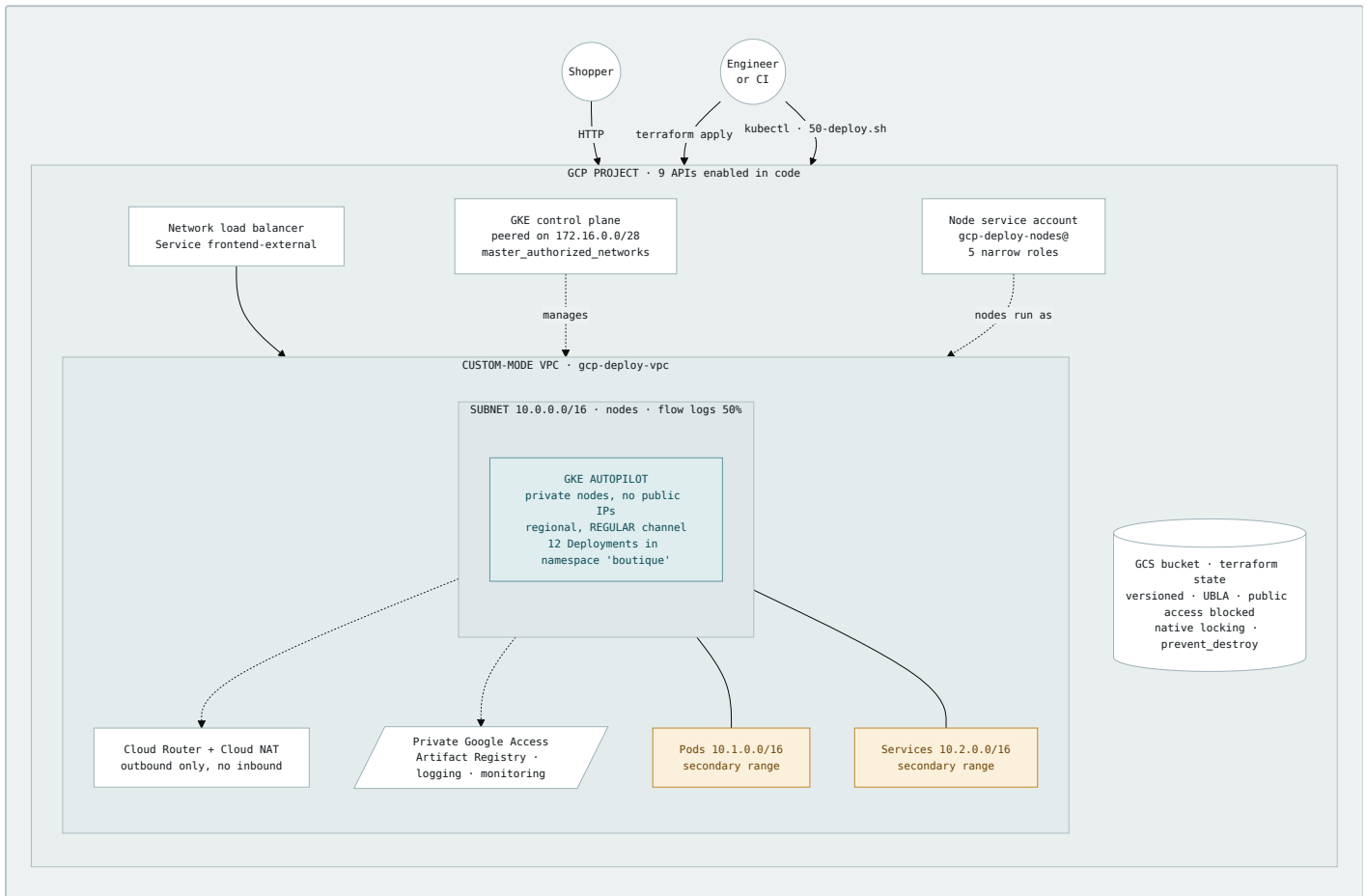
Who uses the system, and what it depends on that we do not own.



The only dependency outside Google Cloud is GitHub, and it is used exactly once per deployment to fetch a pinned manifest. Nothing at runtime depends on it.

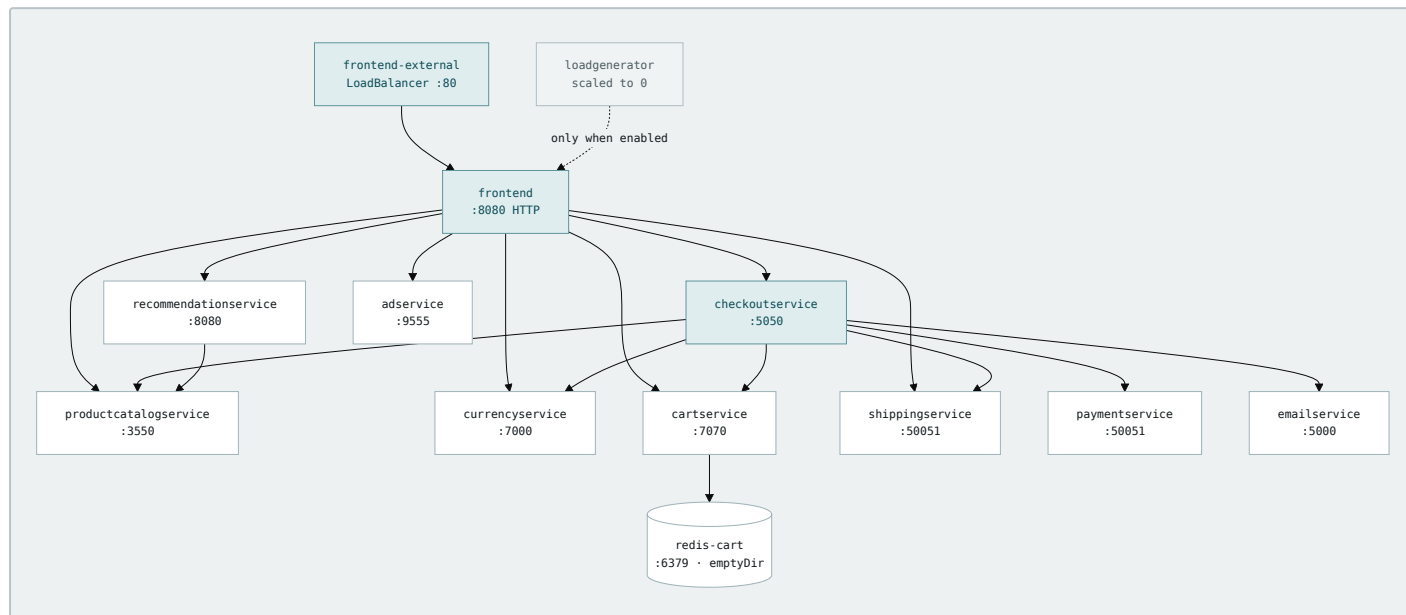
Level 2 – infrastructure containers

What Terraform creates inside the project boundary. The only inbound path to a workload is the load balancer the frontend Service asks for; everything else the cluster needs, it reaches outbound.



Level 3 – application components

The call graph inside the cluster, taken from the `*_SERVICE_ADDR` environment variables in the pinned manifest rather than from the upstream diagram — so it reflects what is actually deployed. All service-to-service traffic is gRPC; only the shopper's traffic is HTTP.



`frontend` and `checkoutservice` are the two fan-out points, and `productcatalogservice` is the most depended-upon service — three callers. That is useful to know before adding `NetworkPolicies` ([03-security](#)), because those edges are the allow-list.

What lands in the cluster

Counted from the pinned `v0.10.6` manifest, applied into the `boutique` namespace — workloads never run in `default`.

OBJECT	COUNT	NOTES
Deployments	12	11 microservices plus <code>redis-cart</code>
Services	12	11 <code>ClusterIP</code> plus <code>frontend-external</code> , the one <code>LoadBalancer</code>
ServiceAccounts	11	One per microservice; <code>redis-cart</code> runs as <code>default</code>
NetworkPolicies	0	None upstream — see 03-security

Address plan

One subnet, three ranges, bound to the cluster through `ip_allocation_policy` — that binding is what makes the cluster VPC-native rather than routes-based.

RANGE	CIDR	ORIGIN	WHERE IT LIVES
Nodes	10.0.0.0/16	Mandated	Primary range of <code>google_compute_subnetwork.gke</code>
Pods	10.1.0.0/16	Mandated	Secondary range <code>Pods</code> → <code>cluster_secondary_range_name</code>
Services	10.2.0.0/16	Mandated	Secondary range <code>services</code> → <code>services_secondary_range_name</code>
Control plane	172.16.0.0/28	<i>Chosen</i>	<code>private_cluster_config.master_ipv4_cidr_block</code> , non-overlapping

The sizing is dramatically larger than the workload needs, implemented as specified, with the disagreement recorded rather than silently corrected — see [ADR 0005](#).

Repository layout

A thin root that only wires modules together. Each module is a blast-radius boundary and the unit of reuse when this pattern is stamped out per environment.

```
.
├─ main.tf                # API enablement + module wiring, nothing else
├─ variables.tf           # Mandated CIDRs as defaults, documented
├─ versions.tf            # Pinned providers + GCS backend
├─ outputs.tf             # Cluster coordinates + proof of the node SA
├─ backend.hcl.example    # Bucket name is supplied at init, not in code
├─ bootstrap/             # Run once, local state: the state bucket
├─ modules/
│   └─ network/           # VPC, subnet + secondary ranges, Router, NAT
│       └─ gke/           # Node SA + IAM, private Autopilot cluster
├─ scripts/               # Seven ordered stages – the number is the order
│   └─ lib.sh             # Shared helpers and defaults, sourced by each stage
│   └─ 10-preflight.sh    # Tooling, fmt, validate – no credentials
│   └─ 20-bootstrap.sh    # Project, billing, ADC, APIs, state bucket, init
│   └─ 30-plan.sh         # Writes a reviewable tfplan
│   └─ 40-apply.sh        # Applies exactly that plan
│   └─ 50-deploy.sh       # Mock CD: apply + health wait + smoke test
│   └─ 60-verify.sh       # Asserts every requirement, non-zero on failure
│       └─ 70-destroy.sh  # Teardown: K8s LBs first, then terraform destroy
├─ docs/                  # These documents; decisions/ holds the ADRs
└─ tools/make-pdf.js      # Renders docs/ into solution-architecture.pdf
```

PATH	WHAT IT DOES
<code>main.tf</code>	Enables 9 APIs with <code>disable_on_destroy = false</code> , then wires <code>network</code> → <code>gke</code> . Destroy removes our resources, not the project's capabilities.
<code>variables.tf</code>	Mandated CIDRs as defaults, each annotated.
<code>versions.tf</code>	Pins Terraform <code>>= 1.10</code> and google <code>~> 6.0</code> ; declares the <code>gcs</code> backend with prefix <code>gcp-deploy/root</code> . The bucket name stays out of code because it is project-specific.
<code>outputs.tf</code>	Cluster name and location, the node SA as proof we are off the Compute default, and a ready-made <code>get-credentials</code> command.
<code>bootstrap/main.tf</code>	Breaks the chicken-and-egg: Terraform needs a remote backend, but something must create the bucket. Versioning, uniform bucket-level access, enforced public-access prevention, <code>prevent_destroy</code> , <code>keep-last-10-versions</code> .
<code>modules/network/main.tf</code>	Custom-mode VPC (auto-mode would collide with the mandated plan), subnet with two secondary ranges, Private Google Access, flow logs at 50% sampling, Cloud Router + NAT logging errors only.
<code>modules/gke/main.tf</code>	Node SA and its five role bindings, then the private regional Autopilot cluster on the REGULAR release channel, with an optional Binary Authorization block.
<code>scripts/lib.sh</code>	Helpers and shared defaults sourced by every stage. Cluster name, region and namespace live here once rather than in three copies that drift.
<code>scripts/10-preflight.sh</code>	Tooling check, <code>fmt -check</code> , <code>validate</code> on both root modules. Needs no credentials, so it runs straight after a clone.
<code>scripts/20-bootstrap.sh</code>	Auth, project, billing, ADC, the two bootstrap APIs, the state bucket, and <code>terraform init</code> against it. Idempotent throughout.
<code>scripts/30-plan.sh</code>	Writes <code>tfplan</code> . Separate from apply so the reviewed change and the applied change are the same one — ADR 0008 .
<code>scripts/40-apply.sh</code>	Applies the saved plan. Takes no <code>PROJECT_ID</code> : the variables are baked into the plan file.
<code>scripts/50-deploy.sh</code>	Fail-fast, non-interactive, idempotent <code>kubectl apply</code> , health-gated <code>kubectl wait</code> , real HTTP smoke test, manifests pinned to a release tag.
<code>scripts/60-verify.sh</code>	Asserts the mandated ranges are bound to the cluster, nodes are private and carry the dedicated service account, every Deployment is Available and the frontend answers. Exits non-zero on any failure.
<code>scripts/70-destroy.sh</code>	Deletes the namespace before Terraform runs, so load balancer forwarding rules created outside Terraform's knowledge cannot block VPC deletion.

Failure modes

What breaks, what the blast radius is, and how it presents. Operational responses are in [04-runbook](#).

FAILURE	BLAST RADIUS	HOW IT PRESENTS
Zone loss	None expected	Regional cluster; control plane and workloads span three zones. Pods reschedule.
Region loss	Total outage	Single region by design — see non-goals . Recovery is re-apply elsewhere, which is fast because it is all code.
Control plane unreachable	Management only	Authorized-network misconfiguration or a private-endpoint switch. Running workloads keep serving; <code>kubectl</code> and <code>terraform</code> stop working.
Cloud NAT port exhaustion	Egress-wide	Image pulls on new nodes fail and outbound calls time out. NAT logging is <code>ERRORS_ONLY</code> , so it shows up there first.
<code>redis-cart</code> Pod restart	All carts emptied	<code>emptyDir</code> ; upstream's design. Nothing alerts on it.
Autopilot scale-up delay	Latency on burst	Pods sit <code>Pending</code> while a node is provisioned — the trade for not managing node pools.
Regional quota exhausted	Apply fails	Trial accounts hit <code>CPUS</code> or <code>IN_USE_ADDRESSES</code> first. No partial-cluster state; re-apply after a quota bump.
State lock left held	All applies blocked	A crashed apply leaves the GCS lock object. Deliberate: blocking is the safe failure. Cleared with <code>terraform force-unlock</code> .
<code>terraform destroy</code> run before the namespace is deleted	Orphaned forwarding rules	Precisely what <code>70-destroy.sh</code> sequences around; run out of order and the VPC delete fails.

Security posture

What this environment defends against, how, and — stated as plainly — where it does not. The gaps are listed because an undisclosed gap is worth less than a disclosed one.

Threat model

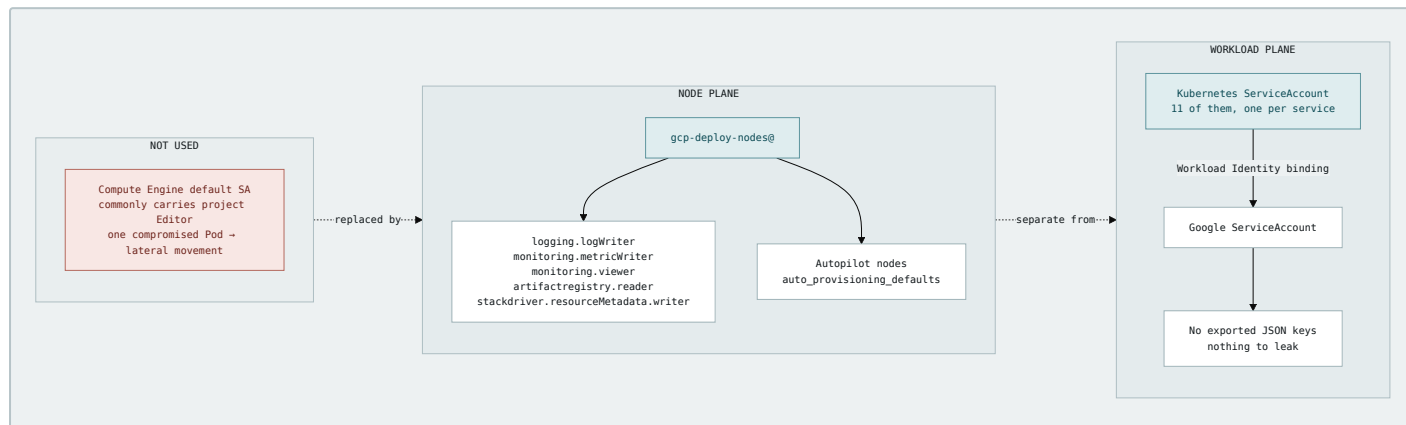
Informal, but explicit about scope. The realistic adversary here is someone who achieves code execution inside a Pod, either through an application vulnerability or a compromised dependency.

IN SCOPE	CONTROL
Compromised Pod pivoting to the cloud project	Dedicated node service account with 5 narrow roles; no <code>roles/editor</code> anywhere in the identity chain — ADR 0003
Direct inbound access to workloads	Private nodes, no external IPs; the only ingress is a load balancer we deliberately create — ADR 0002
Node compromise via the host	Autopilot: no SSH, no privileged Pods, Shielded nodes, Google-managed patching on the REGULAR channel
Credential theft from the filesystem	No service-account JSON keys exist anywhere in the design
Terraform state disclosure	Uniform bucket-level access, enforced public-access prevention, versioning
Concurrent-apply corruption	Native GCS state locking — ADR 0004

EXPLICITLY OUT OF SCOPE	WHY
Malicious insider with project IAM	No separation-of-duties model, no break-glass, no Access Approval
Application-layer attack (XSS, injection, cart tampering)	Upstream demo code, applied unmodified — see non-goals
DDoS	No Cloud Armor; the demo front door is a plain L4 load balancer
Supply-chain compromise of upstream images	Mitigated only partially — see below
Data exfiltration via egress	Cloud NAT permits all outbound; no VPC Service Controls, no egress filtering

Identity: two planes

The single highest-leverage decision is refusing a default. Out of the box, GKE nodes run as the Compute Engine default service account, which commonly carries project Editor — so one compromised Pod inherits near-project-admin lateral movement.



The node plane gets exactly the five roles Google documents as the minimum for a functioning node — logs, metrics, monitoring read, image pull, resource metadata — and nothing else.

IMPORTANT

The workload plane shows the mechanism, not a configured binding

Autopilot enables Workload Identity, and the upstream manifests create a Kubernetes ServiceAccount per service. **This deployment binds none of them to a Google service account, because Online Boutique never calls a Google API.**

That is the honest position: the plane exists and is ready, and the moment a workload needs Cloud SQL or Pub/Sub, the binding is where its credentials come from — never an exported JSON key. If someone asks to see a binding, there isn't one, and there shouldn't be.

Network posture

CONTROL	STATE	DETAIL
Node external IPs	None	<code>enable_private_nodes = true</code>
Inbound	One path	<code>frontend-external</code> , a <code>LoadBalancer</code> Service created by the manifest
Egress	NAT only	Cloud Router + Cloud NAT, <code>ERRORS_ONLY</code> logging
Google API access	Private	Private Google Access on the subnet — image pulls and telemetry never traverse the public internet
Control plane	Public, filtered	<code>master_authorized_networks</code> , currently <code>0.0.0.0/0</code> — ADR 0002
East-west	Unrestricted	No <code>NetworkPolicy</code> — see below
Observability	Sampled	VPC flow logs at 50%, 5-minute aggregation, all metadata

Workload hardening

Inherited from upstream and worth crediting, because it is why the manifests apply cleanly under Autopilot's restrictions with no patching. Every container in the pinned manifest runs:

- `runAsNonRoot: true, runAsUser: 1000, fsGroup: 1000`
- `readOnlyRootFilesystem: true`
- `allowPrivilegeEscalation: false, privileged: false`
- `capabilities: drop: [ALL]`

Autopilot would reject privileged Pods, host mounts and host networking regardless — the platform enforces the floor rather than trusting the manifest to.

Supply chain

LINK	STATE
Manifest provenance	Pinned to release tag <code>v0.10.6</code> rather than a branch, so the same command deploys the same objects — ADR 0007
Image provenance	Upstream published images, referenced by tag. Not by digest, and not signed
Admission control	Binary Authorization wired but disabled, because unsigned images would be rejected — ADR 0006
Vulnerability scanning	None. Artifact Analysis is the paid answer
Provider integrity	<code>.terraform.lock.hcl</code> pins provider checksums across machines and CI

The honest summary: the *delivery path* is reproducible, the *artefacts* are trusted because Google publishes them, not because anything verifies them.

Known gaps

Ranked by what I would fix first.

1. **No NetworkPolicy.** Every Pod can reach every other Pod, so a compromised `adservice` can talk to `redis-cart` directly. Default-deny per namespace plus allow-rules along the edges in the [component diagram](#) is the fix, and that diagram exists partly to make writing them mechanical.
2. **master_authorized_networks is 0.0.0.0/0.** Reachability is still gated by IAM, so this is exposure rather than access — but it is exposure that exists only to keep a demo reproducible.
3. **No admission control in force.** Anything with cluster write access can deploy any image.
4. **Images by tag, not digest.** Tags are mutable; a digest would make the deployment bit-identical over time.
5. **No egress restriction.** A compromised Pod can reach the internet through NAT. VPC Service Controls or an egress proxy is the answer where exfiltration is a stated concern.
6. **deletion_protection = false** on the cluster, for demo teardown. It stays on in production and deletion goes through change control.

Every one of these is a deliberate trade for reviewability, recorded here rather than discovered later. Production paths for all of them are in [06-roadmap](#).

Runbook

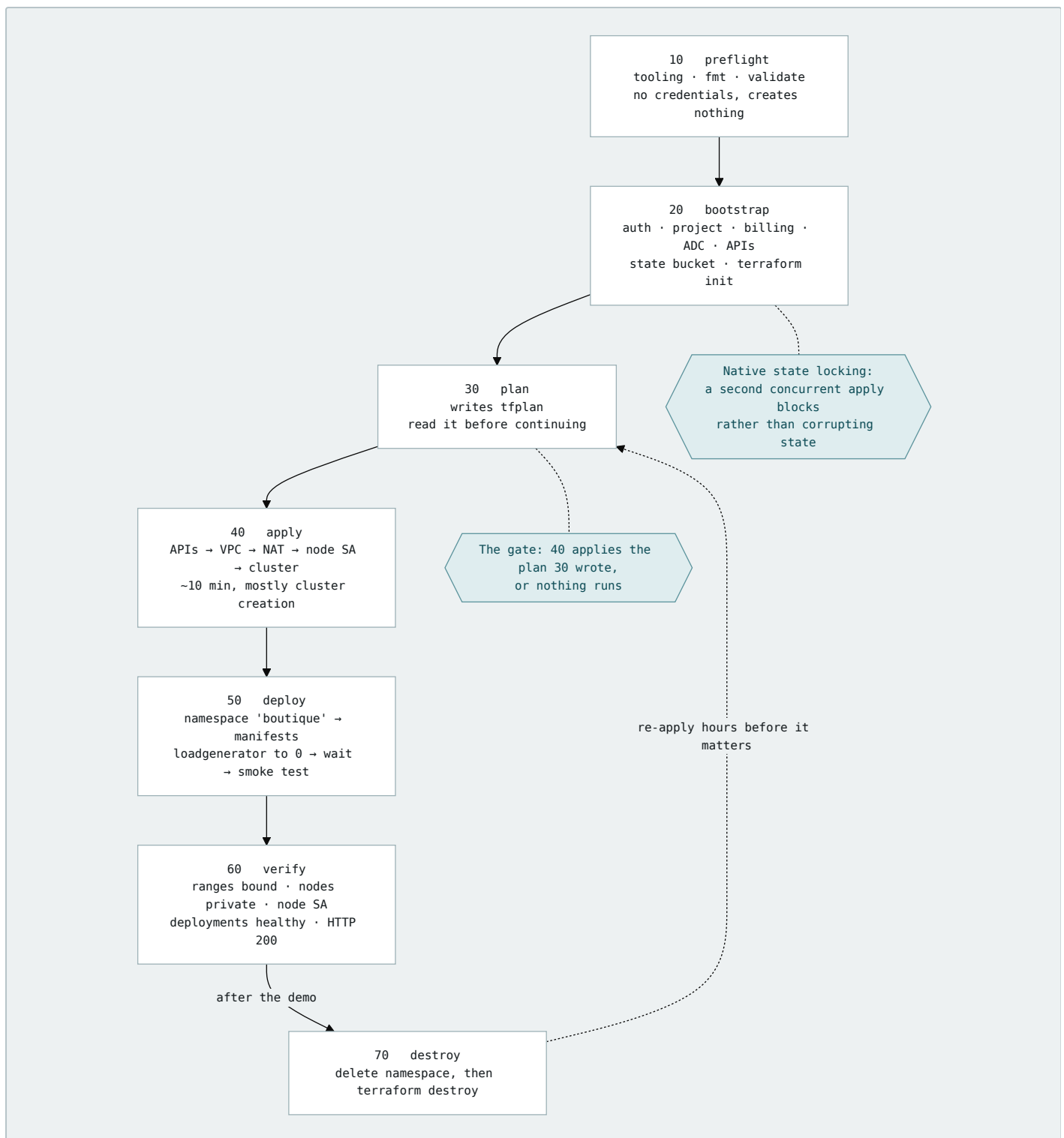
Everything needed to take an empty project to a working shop and back again. This document is task-oriented — the reasoning behind these steps lives in [02-architecture](#) and [decisions/](#).

The stages

Seven numbered scripts under `scripts/`. The number is the order; there is no runner and no build tool, so a person, a Cloud Build step, a GitHub Actions job or an Argo hook can drive them without anything being ported first.

STAGE	SCRIPT	DOES	LIFECYCLE PHASE
10	<code>10-preflight.sh</code>	Tooling, formatting, <code>terraform validate</code>	Test — no credentials, creates nothing
20	<code>20-bootstrap.sh</code>	Auth, project, billing, ADC, APIs, state bucket, <code>init</code>	Provision the ground floor
30	<code>30-plan.sh</code>	Writes a reviewable <code>tfplan</code>	Release gate — ADR 0008
40	<code>40-apply.sh</code>	Applies exactly that plan	Deploy — infrastructure
50	<code>50-deploy.sh</code>	Manifests, health gate, smoke test	Deploy — application
60	<code>60-verify.sh</code>	Asserts every requirement, exits non-zero on failure	Test and monitor
70	<code>70-destroy.sh</code>	Namespace first, then <code>terraform destroy</code>	Operate — teardown

Every stage that needs a project takes `PROJECT_ID` as its first argument — 10 takes none because it touches nothing, 40 takes none because the plan file already carries the variables, and 20 also takes the billing account. Each is safe to re-run and exits non-zero on failure. That uniformity is what makes them a pipeline without a pipeline tool.



Stage 20 runs once per project. The bucket it creates is protected by `prevent_destroy`, so tearing the platform down and rebuilding later resumes at stage 30.

Prerequisites

One thing no script can do for you, because it needs root and differs per distribution:

```
# Arch / CachyOS. Adjust for your distribution.
sudo pacman -S terraform kubectl
paru -S google-cloud-cli google-cloud-cli-gke-gcloud-auth-plugin
```

WARNING

The GKE auth plugin is not optional

Since kubectl 1.26, the kubeconfig written by `gcloud container clusters get-credentials` shells out to `gke-gcloud-auth-plugin`. Without it, every `kubectl` call in `50-deploy.sh` fails on credentials, and the error does not name the missing binary clearly.

Terraform must be `>= 1.10` (`versions.tf`). Stage 10 checks all four binaries and names every one that is missing.

You also need a project that exists and a billing account ID. Stage 20 links the billing account but will not create the project — creation depends on org policy, folder placement and quota that vary per account, so it reports the command instead of guessing.

```
gcloud projects create your-project-id      # if it does not exist yet
gcloud billing accounts list                # to find the billing account ID
```

Deploy

```
export PROJECT_ID=your-project-id
# fish: set -x PROJECT_ID your-project-id

./scripts/10-preflight.sh
./scripts/20-bootstrap.sh "$PROJECT_ID" XXXXXX-XXXXXX-XXXXXX
./scripts/30-plan.sh "$PROJECT_ID"          # read the plan before continuing
./scripts/40-apply.sh                       # ~10 min
./scripts/50-deploy.sh "$PROJECT_ID"
./scripts/60-verify.sh "$PROJECT_ID"
```

The scripts resolve the repository root from their own location, so they work from any directory.

Stage 50 prints the shop's URL, having already checked it returns HTTP 200. Stage 60 then asserts every requirement independently and fails loudly if one does not hold.

Useful variations:

```
# Run the load generator for a live autoscaling demo (costs money while up)
DEPLOY_LOADGENERATOR=true ./scripts/50-deploy.sh "$PROJECT_ID"
# fish: env DEPLOY_LOADGENERATOR=true ./scripts/50-deploy.sh $PROJECT_ID

# Try a different upstream release
MANIFEST_VERSION=v0.10.5 ./scripts/50-deploy.sh "$PROJECT_ID"
# fish: env MANIFEST_VERSION=v0.10.5 ./scripts/50-deploy.sh $PROJECT_ID

# Lock the control plane to your own address instead of 0.0.0.0/0
terraform plan -out=tfplan -var="project_id=$PROJECT_ID" \
  -var='authorized_networks=[{"cidr_block="203.0.113.4/32",display_name="office"}]
terraform apply tfplan
```

Why the stages are cut where they are

Each boundary is a change in what can go wrong, not a narrative convenience.

10 needs nothing. No credentials, no project, no billing. It runs the moment someone clones, so a broken checkout costs seconds rather than eight minutes into a cluster create. `terraform init -backend=false` is what makes validation possible before the state bucket exists.

20 is the only stage that creates something cheap. A backend cannot create its own bucket, so `bootstrap/` runs with local state and makes it. Stage 20 then writes `backend.hcl` and initialises the root module against it. The bucket costs pennies and never gets destroyed.

FAILURE

Two different credentials

`gcloud auth login` authenticates the CLI. Terraform reads Application Default Credentials, which is separate. Stage 20 sets up both because skipping the second is the single most common first-apply failure.

Terraform also needs Service Usage enabled before it can enable the other nine APIs in code, which is why stage 20 turns that one on with `gcloud` first.

30 and 40 are separate on purpose.

IMPORTANT

Why the plan is saved rather than confirmed at the prompt

Bare `terraform apply` prints a plan and waits for `yes`, but it re-plans at the moment you answer — so what executes is not strictly what you read. `apply tfplan` executes the reviewed plan or fails.

The saved plan is also the only artefact this repository produces that can be reviewed *before* the change: attachable to a pull request, readable by someone who is not at the keyboard. Until [real CI](#) provides a pipeline gate, this is the gate. `tfplan` and `*.tfplan` are both gitignored — plans can embed sensitive values.

60 exists so the claims are checkable by machine. Every other document in this repository asserts that the requirements are met. Stage 60 proves it, and returns a non-zero exit code when it cannot.

Proving it works

```
./scripts/60-verify.sh "$PROJECT_ID"
```

Every mandated requirement asserted, grouped by requirement, each mapping to a success criterion in [01-context](#). It reports everything wrong in one pass rather than stopping at the first failure.

Two are worth understanding rather than trusting:

The ranges are bound, not merely present. A subnet can carry `10.1.0.0/16` while the cluster ignores it. Stage 60 checks the CIDRs on the subnet *and* that `ipAllocationPolicy` points at them, which is what makes the cluster VPC-native rather than routes-based.

Nodes actually run as the dedicated service account. `terraform output node_service_account` proves the account was created, nothing more. Stage 60 reads `autoscaling.autoprovisioningNodePoolDefaults.serviceAccount` off the cluster — the

account Autopilot actually provisions nodes with — because Autopilot does not surface its node VMs as listable Compute instances. That is the failure [ADR 0003](#) warns about: omit the Autopilot wiring and you get a cluster that looks least-privilege while its nodes use the Compute Engine default.

The same checks by hand, if you would rather see them individually:

```
gcloud container clusters describe gcp-deploy --region europe-west4 \
  --format='value(privateClusterConfig.enablePrivateNodes,
                  ipAllocationPolicy.clusterSecondaryRangeName,
                  ipAllocationPolicy.servicesSecondaryRangeName)'\

# The account Autopilot provisions nodes with, not the Compute Engine default
gcloud container clusters describe gcp-deploy --region europe-west4 \
  --format='value(autoscaling.autoprovisioningNodePoolDefaults.serviceAccount)'\

# Nodes carry no external IPs. Autopilot node VMs are not listable Compute
# instances, so ask the Kubernetes API instead – any output at all is a finding.
kubectl get nodes \
  --output=jsonpath='{range .items[*]}.{.status.addresses[?(@.type=="ExternalIP")].address}{"\n"}{end}\'

# The roles that account actually holds – expect exactly five
gcloud projects get-iam-policy "$PROJECT_ID" \
  --flatten='bindings[].members' \
  --filter="bindings.members:gcp-deploy-nodes@" \
  --format='value(bindings.role)'\

# Every workload healthy, in its own namespace
kubectl get pods --namespace boutique

# State locking is real: run this during an apply and watch it block
terraform plan -var="project_id=$PROJECT_ID"
```

Troubleshooting

SYMPTOM	CAUSE	FIX
Error 403: ... API has not been used	API enablement is asynchronous	Re-run stages 30 and 40; both are idempotent
Quota 'CPUS' exceeded / IN_USE_ADDRESSES	Trial-account regional quota	<i>IAM & Admin</i> → <i>Quotas</i> , request an increase, or upgrade the account — credits survive the upgrade
google: could not find default credentials	ADC never created	Re-run stage 20, or <code>gcloud auth application-default login</code>
no Auth Provider found for name "gcp" or a missing plugin	<code>gke-gcloud-auth-plugin</code> absent	Install it, then re-run stage 50
no tfplan found	Stage 40 run before stage 30	Run <code>./scripts/30-plan.sh "\$PROJECT_ID"</code>
Saved plan is stale	State moved between stages 30 and 40	The gate working. Re-plan and read it again
Error acquiring the state lock	A previous apply crashed holding the lock	Confirm nobody else is applying, then <code>terraform force-unlock <LOCK_ID></code>
kubectl times out	Your address is outside <code>master_authorized_networks</code>	Re-plan and apply with your CIDR, or widen it temporarily
Pods stuck Pending	Autopilot is provisioning capacity	Wait; if it persists, check for a resource-ratio rejection in <code>kubectl describe pod</code>
Stage 50 times out waiting for the LB IP	Load balancer provisioning is slow or quota-blocked	<code>kubectl get svc frontend-external --namespace boutique --watch</code>
<code>terraform destroy</code> fails deleting the VPC	Kubernetes-created forwarding rules still exist	Delete the namespace first — which is what stage 70 does

Teardown

```
./scripts/70-destroy.sh "$PROJECT_ID"
```

Order matters: the namespace goes first so that load balancer forwarding rules, created by Kubernetes and invisible to Terraform, are gone before the VPC delete. Watch that step succeed — if it times out the script continues anyway, and the orphaned rules will block the VPC.

The state bucket survives by design (`prevent_destroy`), as do the enabled APIs, so the next run resumes at stage 30.

Confirm nothing is left:

```
gcloud container clusters list
gcloud compute instances list
gcloud compute forwarding-rules list
gcloud compute addresses list
```

Tear down after every working session. Cloud NAT and the load balancer bill while idle whether or not anyone is shopping — see [05-cost](#).

Cost model

What this environment bills for, computed from the manifest that is actually deployed rather than estimated by feel — and where the numbers would change with a production budget.

WARNING

Rates are indicative, the arithmetic is not

List prices move and vary by region. Every figure below shows its working, so substitute the current `europa-west4` rate from the [pricing calculator](#) and the conclusion still holds. The *shape* of the bill — which line items dominate, and which are fixed versus variable — is what this document is for.

For actuals rather than estimates: *Billing* → *Reports*, grouped by SKU.

What Autopilot actually charges for

Autopilot bills per **Pod resource request**, not per node — that is the cost argument for it ([ADR 0001](#)). It also enforces per-Pod minimums and rounds requests up, so the manifest's numbers are a floor you do not get to keep.

Summed from the pinned `v0.10.6` manifest, with `loadgenerator` at zero replicas:

DEPLOYMENT	CPU REQUEST	MEMORY REQUEST
<code>adservice</code>	200m	180 Mi
<code>cartservice</code>	200m	64 Mi
<code>checkoutservice</code>	100m	64 Mi
<code>currencyservice</code>	100m	64 Mi
<code>emailservice</code>	100m	64 Mi
<code>frontend</code>	100m	64 Mi
<code>paymentservice</code>	100m	64 Mi
<code>productcatalogservice</code>	100m	64 Mi
<code>recommendationservice</code>	100m	220 Mi
<code>redis-cart</code>	70m	200 Mi
<code>shippingservice</code>	100m	64 Mi
As requested	1.27 vCPU	1.09 GiB
As billed (250m / 512Mi per-Pod minimums)	2.75 vCPU	5.50 GiB

Autopilot bills 2.2× the CPU and 5.1× the memory these manifests ask for. Eleven small services each request far less than the platform's minimum, and each one is rounded up individually. That is the single most useful cost fact about this deployment, and it is invisible unless you go looking — the manifest says 1.09 GiB and the invoice reflects 5.5.

The lesson generalises: on Autopilot, *many tiny Pods* is an anti-pattern. Consolidation, or accepting a larger request you actually use, costs the same or less.

Estimated monthly bill

At 730 hours, everything left running continuously — which you should not do.

LINE ITEM	BILLING BASIS	INDICATIVE RATE	MONTHLY
Autopilot vCPU	2.75 vCPU × 730 h	~\$0.0445 / vCPU-h	~\$89
Autopilot memory	5.50 GiB × 730 h	~\$0.0049 / GiB-h	~\$20
Cluster management fee	1 cluster × 730 h	~\$0.10 / h	~\$73
Cloud NAT gateway	1 gateway × 730 h	~\$0.044 / h	~\$32
Cloud NAT data processing	egress volume	~\$0.045 / GiB	negligible here
Load balancer forwarding rule	1 rule × 730 h	~\$0.025 / h	~\$18
VPC flow logs	50% sampled	per GiB ingested	pennies at this scale
GCS state bucket	kilobytes, versioned	per GiB-month	pennies
Total			~\$232

The free tier covers one cluster's management fee, taking it to roughly **\$159/month** left running.

The number that matters more

SCENARIO	COST
A four-hour demo session	~\$1.30
Left running for a month	~\$159–232
Infrastructure up, app deleted	~\$105/month (cluster fee + NAT; the LB goes with the namespace)

Leaving it up for a month costs over a hundred times what actually demonstrating it costs. That asymmetry is the entire justification for `70-destroy.sh` and for treating teardown as part of the workflow rather than housekeeping.

Cost controls already in the repository

CONTROL	WHERE	SAVES
loadgenerator scaled to 0 by default	50-deploy.sh	300m CPU / 512 Mi billed continuously (256 Mi requested, rounded up to the per-Pod floor) — and it generates NAT-processed traffic
One-command teardown	70-destroy.sh	Everything except the state bucket
Flow logs sampled at 50%	modules/network/main.tf	Half the log ingestion, same forensic value at this scale
NAT logging <code>ERRORS_ONLY</code>	modules/network/main.tf	Logging every translation event would dwarf the signal
Autopilot over Standard	modules/gke/main.tf	No idle node capacity — you pay for Pods, not for headroom

Free tier now, premium in production

Several choices here are deliberately the budget variant of a control with a paid, production-grade counterpart. Being able to name both sides is the point.

AREA	THIS BUILD	PRODUCTION CONSIDERATION	WHY IT MATTERS
Network service tier	Premium (the GCP default)	Premium, non-negotiable — required for the global external ALB with an anycast IP	Standard tier means hot-potato routing and regional-only load balancing
GKE edition	Standard edition on Autopilot	GKE Enterprise for fleet management, Config Sync, Policy Controller, managed service mesh	Per-tenant namespaces × many tenants × policy enforcement is exactly what it is priced for
DDoS / WAF	None	Cloud Armor ; evaluate Enterprise for bill protection during an attack	On per-request billing, a volumetric attack is also a <i>financial</i> attack
Availability	Regional cluster, no SLA on trial	Multi-region behind a global anycast IP; published per-tier SLOs	Free trial buys architectural HA, not a contractual one
Support	Community	Standard → Enhanced → Premium , scaled to revenue at risk	The first Sev-1 pays for a year of Enhanced
Commitment pricing	On-demand, torn down nightly	Committed use discounts once a steady-state profile exists	Commit to the floor, burst on demand
Security tooling	Least privilege, private nodes, BinAuthz off	Security Command Center , Artifact Analysis scanning, BinAuthz enforced	Free tier proves the design; paid tooling provides continuous evidence
Log retention	Defaults, 50% flow sampling	13-month audit retention, sinks to BigQuery	Incident forensics and per-tenant billing both die without history

TIP

Free tier constrains spend, not architecture

Everything structural — private VPC-native cluster, the identity chain, remote state, GitOps-shaped delivery — is identical at both ends. What budget buys is contractual assurance, continuous evidence and scale economics. That also answers the usual follow-up: with real money, nothing changes about the design and a great deal changes about the guarantees around it.

Trial-account fine print

Trial accounts carry reduced regional quotas and cannot claim SLAs. An `apply` that fails on quota is almost always regional `CPUS` or `IN_USE_ADDRESSES`, both visible under *IAM & Admin* → *Quotas* and both resolved by upgrading the account — credits remain usable afterwards.

Roadmap

What I would do next, ranked by value per unit of effort rather than by how impressive it sounds. Each item names the gap it closes, so the list can be argued with rather than just admired.

Next, in order

1. Default-deny NetworkPolicy between the microservices

Closes: [gap 1](#) — every Pod can currently reach every other Pod. **Effort:** hours. The allow-list is already drawn: it is the [component diagram](#), derived from the manifest's own service-address variables. **Why first:** it is the largest reduction in blast radius available for the least work, and it demonstrates east-west thinking rather than perimeter thinking. Autopilot supports NetworkPolicy natively.

2. Lock the control plane down

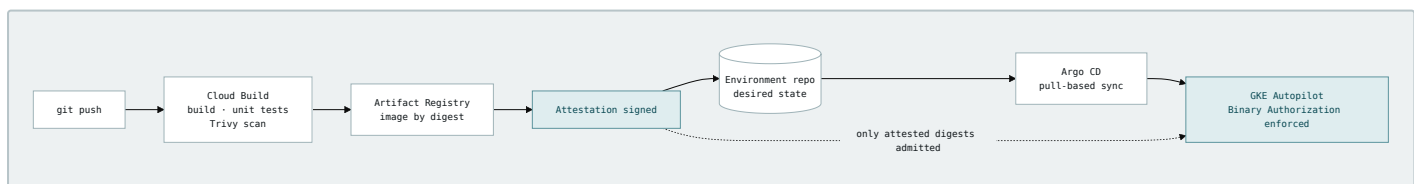
Closes: [gap 2](#) — `master_authorized_networks` is `0.0.0.0/0`. **Effort:** one variable for the CIDR restriction; a day for the full private-endpoint path with IAP tunnelling or Connect Gateway. **Why here:** the demo-friendliness argument in [ADR 0002](#) evaporates the moment this stops being a demo.

3. An `envs/` layer over the same modules

Closes: the [single-environment assumption](#). **Effort:** a day. `envs/dev` and `envs/prod` directories, each with its own backend prefix and tfvars, both consuming `modules/network` and `modules/gke` unchanged. **Why it matters:** it is the proof that the module boundaries were drawn correctly. If a second environment can be added without touching a module, the abstraction held.

4. Real CI/CD, replacing the mock

Closes: the [delivery non-goal](#). **Effort:** days.



Three properties this buys that `50-deploy.sh` cannot: CI never holds cluster credentials (Argo pulls), only attested digests run ([ADR 0006](#) flips to enforced), and CI authenticates to Google through Workload Identity Federation, so no service-account key exists anywhere in the chain.

5. Gateway API behind a global ALB

Closes: the demo `LoadBalancer` front door. **Effort:** a day, plus DNS and certificates. **Brings:** Cloud Armor, CDN, an anycast IP, and TLS terminated at the edge — the front-door-first pattern, and the prerequisite for anything multi-region.

6. Managed Prometheus dashboards and SLOs

Closes: no defined service levels, and no alerting on the [failure modes](#). **Effort:** a day for the obvious four — availability and latency at the frontend, error rate per backend, and an alert on `redis-cart` restarts, which currently empties every cart silently and notifies nobody. **Why:** Autopilot ships Managed Service for Prometheus with no agents to install, so the marginal cost is dashboards and thought rather than infrastructure.

7. Memorystore for the cart

Closes: the [ephemeral](#) `emptyDir`. **Effort:** half a day, plus the first genuine use of a Workload Identity binding — which would finally populate the workload plane in [03-security](#) with something real.

Deliberately not on this list

- **A service mesh.** mTLS and traffic policy are real benefits; a mesh control plane for eleven services that already work is not yet justified. Revisit at multi-tenancy.
- **Multi-region.** Buy the SLA and the global front door first. Multi-region without a global load balancer and a data replication story is theatre.
- **Autoscaling tuning.** No production load profile exists to tune against. Guessing would produce numbers that look authoritative and mean nothing.

How the documentation scales

This repository documents itself at the size it is. At team scale the same structure grows rather than changes:

- **ADRs keep accumulating.** They are immutable — a reversal is a new record with `Supersedes 0002`, never an edit. The history of what was believed when is the point.
- **CI regenerates the PDF and checks links.** Today `tools/make-pdf.js` is run by hand, which means it can drift. It should run on every merge to `main`.
- **One ADR per pull request that changes a decision.** Cheapest possible way to keep the "why" current, because it happens while the reasoning is still in someone's head.

Architecture decision records

One file per decision, in [Michael Nygard's format](#). Each records **what was decided, when, why, and what it cost** — including the alternatives that lost and the reason they lost.

Index

#	DECISION	STATUS
0001	GKE Autopilot as the compute platform	Accepted
0002	Private nodes with a guarded public control-plane endpoint	Accepted
0003	A dedicated node service account, not the Compute default	Accepted
0004	Versioned GCS remote state with native locking	Accepted
0005	Implement the mandated CIDR sizing, and record the disagreement	Accepted
0006	Binary Authorization wired but disabled by default	Accepted
0007	Pin upstream manifests to a release tag, not a branch	Accepted
0008	An explicit saved-plan gate before apply	Accepted

Conventions

Decisions are not rewritten. A decision that gets reversed is not edited — a new record supersedes it, and the old one gains `Superseded by 00NN`. The value of an ADR is that it shows what was believed *at the time*, and editing that away destroys the thing worth keeping.

Facts are corrected in place. A dead link, a resource that has been renamed, or a consequence that later work has closed gets fixed where it stands. Leaving a record that describes the repository incorrectly helps nobody, and correcting a fact is a different act from rewriting a judgement.

Statuses: `Proposed` → `Accepted` → `Superseded by 00NN` (or `Deprecated` where nothing replaces it).

One decision per record. If a record needs the word "and" in its title, it is probably two records.

Consequences include the negative ones. A record listing only benefits is marketing. The negative consequences are the part a future reader actually needs, because they are what they will be living with.

New records are numbered sequentially and never renumbered, so links and references stay stable.

ADR 0001 – GKE Autopilot as the compute platform

- **Status:** Accepted
- **Date:** 2026-08-03
- **Deciders:** Platform engineering

Context

The brief requires a private, least-privilege Kubernetes environment running Online Boutique, built entirely as code, on an account backed by trial credits. Two forces pull hardest: *least privilege must be structural rather than a checklist*, and *resource utilisation should be optimised for cost*.

Online Boutique ships as first-class Kubernetes manifests — 12 Deployments, a gRPC mesh between them, and an in-cluster Redis. Nothing in it needs node-level agents, custom machine shapes, GPUs, or host access.

Decision

Use **GKE Autopilot**, regional, on the REGULAR release channel.

Consequences

Positive

- Billing is per Pod resource request, so there is no idle node capacity to pay for. Cost tracks what is actually asked for.
- Security posture is opinionated by default: no SSH to nodes, no privileged Pods, Shielded nodes always on, node patching handled by Google. Least privilege becomes a property of the platform rather than a thing we remember to configure.
- Node pool sizing, bin-packing and upgrades stop being our problem, which is the largest single reduction in operational surface available.
- Workload Identity is enabled by default, so the workload identity plane exists without extra work ([03-security](#)).

Negative

- Per-Pod minimums (250m CPU / 512 MiB) are enforced and applied *per Pod*. With eleven small services this bills 2.2× the CPU and 5.1× the memory the manifests request — quantified in [05-cost](#). Many-small-Pods is an anti-pattern here.
- No DaemonSets requiring host access, no privileged sidecars, no custom kernel parameters. If a future requirement needs any of these, the escape hatch is a migration, not a flag.
- Pods can sit `Pending` while Autopilot provisions capacity — burst latency is the trade for not managing node pools.
- Less control over node placement and machine families than Standard.

Alternatives considered

GKE Standard. More knobs: custom node pools, DaemonSet freedom, unusual GPU configurations, precise machine-type control. Rejected because nothing in the workload requires them and every one of those knobs is also a way to weaken the security posture by accident. It remains the escape hatch if node-level agents become a requirement.

Cloud Run. Lower operational overhead still, and attractive for stateless HTTP services. Rejected because the application is distributed as Kubernetes manifests with a gRPC service mesh and an in-cluster Redis; porting it would mean modifying the application, which is an explicit [non-goal](#). It would also diverge from a per-tenant-namespaces direction of travel.

Compute Engine managed instance groups. Rejected outright: lift-and-shift would relocate the VM-sprawl problem rather than escape it, and would put node patching, image building and OS hardening back on us — the opposite of the least-privilege objective.

ADR 0002 – Private nodes with a guarded public control-plane endpoint

- **Status:** Accepted
- **Date:** 2026-08-03
- **Deciders:** Platform engineering

Context

Least privilege applies to network reachability as much as to IAM. Two separate exposure questions get conflated routinely, and they have different answers:

1. **Can anything reach the workloads directly?** This should be no, with one deliberate exception.
2. **Can an operator reach the Kubernetes API?** This must be yes, from somewhere.

The environment is reviewed by a panel who need to reproduce it from arbitrary networks, and one of the stated [assumptions](#) is that a reviewer's source address is not known in advance.

Decision

- **Nodes are private:** `enable_private_nodes = true`, no external IPs on any node.
- **Egress is NAT-only:** Cloud Router plus Cloud NAT, with Private Google Access on the subnet so Google API traffic never leaves Google's network.
- **The control-plane endpoint stays public** (`enable_private_endpoint = false`) but is filtered by `master_authorized_networks`, currently `0.0.0.0/0` with the demo intent recorded in the variable's description.
- **Inbound to workloads exists only where explicitly created** — the `frontend-external` LoadBalancer Service.

Consequences

Positive

- No workload is directly reachable from the internet. The attack surface is one load balancer, deliberately created.
- Image pulls and telemetry travel over Private Google Access rather than the public internet.
- Egress is centralised through NAT, so it is observable in one place and could later be filtered in one place.
- A reviewer can clone, apply and `kubectl` from anywhere with no bastion, which keeps the demo reproducible.

Negative

- `0.0.0.0/0` on the control plane is exposure that exists purely for demo convenience. Access is still gated by IAM — an unauthenticated caller gets nothing —

but the endpoint is discoverable and reachable, which is strictly worse than not being.

This is recorded as [gap 2](#) and item 2 on the [roadmap](#).

- Cloud NAT is a fixed monthly cost that accrues whether or not anyone is using the environment — roughly a fifth of the idle bill ([05-cost](#)).
- NAT port exhaustion becomes a cluster-wide egress failure mode ([02-architecture](#)).

Alternatives considered

Fully private endpoint with a bastion host. The stronger posture, and the production answer. Rejected for this deliverable because it requires either a bastion VM (another machine to patch, and a standing credential target) or IAP tunnelling configuration, both of which add reviewer friction for no gain in what is being demonstrated.

Private endpoint plus Connect Gateway. The best production option — no bastion, IAM-mediated, works from anywhere. Rejected only because it is more moving parts than a demo justifies. This is the recommended path forward.

Restricting `master_authorized_networks` to a known CIDR. Correct in any real engagement, and supported today by passing the variable — the runbook [shows the command](#). Not the default only because the reviewer's address is unknown.

Public nodes. Simpler, and rejected immediately: it trades a large permanent increase in attack surface for a small reduction in setup complexity.

ADR 0003 – A dedicated node service account, not the Compute default

- **Status:** Accepted
- **Date:** 2026-08-03
- **Deciders:** Platform engineering

Context

If no service account is specified, GKE nodes run as the **Compute Engine default service account**. On many projects that account carries `roles/editor`, because it is granted that by default when the Compute API is enabled and it is rarely trimmed afterwards.

The consequence is specific and severe: any process that achieves code execution in any Pod can reach the node's metadata server, obtain that account's token, and act as project Editor. One application vulnerability becomes near-total project compromise, with no further exploitation required.

This is the highest-severity default in a standard GKE deployment, and it is silent — nothing warns you, and the cluster works perfectly either way.

Decision

Create a dedicated service account, `gcp-deploy-nodes@`, and bind exactly the five roles Google documents as the minimum for a functioning node:

ROLE	NEEDED FOR
<code>roles/logging.logWriter</code>	Shipping node and Pod logs
<code>roles/monitoring.metricWriter</code>	Shipping metrics
<code>roles/monitoring.viewer</code>	The metrics agent reading back
<code>roles/artifactregistry.reader</code>	Pulling images
<code>roles/stackdriver.resourceMetadata.writer</code>	Resource metadata for observability

Attach it to Autopilot's node provisioning via `cluster_autoscaling.auto_provisioning_defaults.service_account`, so nodes created on demand inherit it rather than falling back to the default.

Keep this strictly separate from **workload** identity: applications that need Google APIs get a Workload Identity binding between their Kubernetes ServiceAccount and a Google service account. Node credentials are never the mechanism by which an application authenticates, and no JSON key is ever exported.

Consequences

Positive

- A compromised Pod inherits log-write, metric-write and image-pull. There is no path from there to project administration.
- The blast radius of the node identity is now small enough to state in one sentence, which is what makes it auditable.
- `terraform output node_service_account` makes the claim checkable in seconds ([04-runbook](#)).
- The two identity planes stay conceptually distinct, so nobody is tempted to "just add a role to the node SA" when an application needs an API.

Negative

- Any future node-level agent needing a further permission requires a deliberate role addition — which is the point, but it is friction.
- Five bindings are five things to keep correct; drift here is silent until something stops shipping telemetry.
- Autopilot's `auto_provisioning_defaults` is easy to omit. Creating the account without wiring it there produces a cluster that *looks* least-privilege while nodes still run as the default. The `depends_on` in `modules/gke/main.tf` exists so the bindings land before the cluster does.

Alternatives considered

Use the Compute Engine default service account. Zero effort, and the reason this misconfiguration is so widespread. Rejected — it is the specific failure this decision exists to prevent.

Use the default account but strip `roles/editor` from it. Better, but project-wide: it affects every resource that account backs, so the change is riskier than creating a purpose-built account, and it leaves a shared identity across unrelated workloads.

Grant the node account nothing and rely on Workload Identity for everything. Attractive in principle. Rejected because nodes genuinely need to pull images and ship their own telemetry before any workload identity exists — a cluster whose nodes cannot log is not observable.

ADR 0004 – Versioned GCS remote state with native locking

- **Status:** Accepted
- **Date:** 2026-08-03
- **Deciders:** Platform engineering

Context

Requirement [R5](#) — "engineers collaborate without side effects" — is, underneath the phrasing, a **state** problem. With local state, three failures are inevitable rather than possible:

1. Two engineers apply concurrently; the second overwrites the first's state and the infrastructure no longer matches any record of it.
2. One engineer's laptop holds the only copy of the truth.
3. Different provider versions produce different plans from identical code.

A secondary problem: Terraform needs a backend bucket, but the bucket has to be created by something. Solving that inside the same root module is circular.

Decision

A **separate bootstrap/ root module**, run once with local state, creates the state bucket with:

- object **versioning** — state history and rollback
- **uniform bucket-level access** — IAM only, no per-object ACLs
- **public access prevention: enforced**
- **prevent_destroy** — Terraform refuses to delete it
- a lifecycle rule keeping the last 10 noncurrent versions

The root module then uses that bucket as a `gcs` backend with prefix `gcp-deploy/root`. The bucket *name* is supplied at init time via `-backend-config`, never hardcoded, because it is project-specific. Provider versions are pinned (`~> 6.0`) and `.terraform.lock.hcl` is committed.

Consequences

Positive

- The GCS backend provides **native state locking** with no extra infrastructure. A second concurrent apply blocks instead of corrupting state — Terraform's DynamoDB-equivalent comes free on GCS.
- Versioning means a bad state push is recoverable rather than terminal.
- The bootstrap module's own local state becomes irrelevant immediately after the first run; `prevent_destroy` protects the bucket from the module that made it.
- Pinned providers plus a committed lockfile mean every engineer and CI runner resolves byte-identical binaries.
- A fresh clone converges with one `apply`, because API enablement is in code too.

Negative

- The bootstrap module is a second root module to understand, with its own state and its own lifecycle. `20-bootstrap.sh` runs both Terraform steps as one command, so it is no longer a manual prerequisite, but it is still a second thing in the repository that a newcomer has to account for.
- A crashed apply leaves the lock held, and the next engineer sees `Error acquiring the state lock` with no obvious owner. Blocking is the safe failure, but it needs `force-unlock` and the judgement to know when that is safe.
- `backend.hcl` is gitignored, so it never travels with the repository. `20-bootstrap.sh` regenerates it from `backend.hcl.example`, so nobody types a bucket name, but a fresh clone has no backend configuration until that stage has run.
- State still contains resource identifiers and any sensitive outputs. Bucket IAM is the only thing protecting it; there is no state encryption beyond Google-managed keys at rest.

Alternatives considered

Local state. Rejected: fails requirement R5 outright.

Terraform Cloud / HCP Terraform. Adds plan-on-PR, policy checks, a run history and hosted state. The right answer at team scale and where I would go next. Rejected here as more moving parts and an external dependency than a single-project deliverable justifies.

Atlantis. Plan-on-PR without leaving your own infrastructure, at the cost of running and securing Atlantis itself. Same reasoning as above.

A hand-created bucket via `gcloud`. Simpler than a bootstrap module, and rejected because it puts a mandatory manual step outside code — precisely the clickops prerequisite this repository is trying not to have.

ADR 0005 – Implement the mandated CIDR sizing, and record the disagreement

- **Status:** Accepted
- **Date:** 2026-08-03
- **Deciders:** Platform engineering

Context

The brief mandates three ranges: `10.0.0.0/16` for nodes, `10.1.0.0/16` for Pods, `10.2.0.0/16` for Services ([R1–R3](#)).

Two of the three are considerably larger than any plausible need:

- **Nodes: a /16 is 65,536 addresses.** This cluster will run single-digit nodes. A /22 (1,022 usable) would be generous.
- **Services: a /16 is 65,536 ClusterIPs.** The workload defines 12. A /20 would be lavish.
- **Pods: a /16 is defensible.** GKE allocates a /24 per node by default, so a /16 supports 256 nodes — the one range where the mandate is close to reasonable.

VPC address space is painful to reclaim: primary ranges cannot be shrunk in place, so correcting this later means a new subnet and a cluster migration. Oversized ranges also foreclose future peering options — an RFC1918 block consumed here is unavailable to anything that must peer without overlap.

Decision

Implement the mandated ranges exactly as specified. Record the disagreement in `variables.tf`, in this ADR, and in [02-architecture](#) — visible to anyone reading either the code or the documents, and impossible to mistake for an oversight.

Consequences

Positive

- The brief is satisfied literally and verifiably. Compliance is checkable with one `gcloud` command ([04-runbook](#)).
- The reasoning is preserved at the point of use. A future engineer sees the disagreement in the variable definition, not only in a document they may never open.
- The ranges are variables with defaults, not literals, so right-sizing is a `tfvars` change rather than a refactor.

Negative

- Roughly 130,000 RFC1918 addresses are consumed to serve a workload needing a few hundred. In an organisation with an address-plan authority, this would be a rejected allocation request.
- Peering with any network already using `10.0.0.0/14` becomes impossible without renumbering one side.

- It bakes in a habit — "take a /16, it's fine" — that scales badly across many clusters.

Alternatives considered

Right-size them anyway and explain afterwards. Produces a technically better network and fails the requirement as written. A reviewer checking compliance finds a mismatch and has to decide whether it was judgement or carelessness — and they cannot tell from the artefact. Rejected: silently substituting your own judgement for a stated requirement is the actual failure here, and it is worse than the oversized range.

Implement as mandated and say nothing. Rejected for the opposite reason. It reads as not having noticed, which is indistinguishable from not knowing.

Split the difference — mandated for Pods, right-sized for nodes and Services. Rejected as the worst of both: partial compliance, no clean story, and a reviewer still has to work out which parts were deliberate.

Note

This is the pattern I would follow on any engagement: raise the concern once, in writing, with the reasoning and the cost of being wrong; then build what was agreed. The disagreement is documented, not litigated, and if the constraint was arbitrary the record makes it cheap to revisit later.

ADR 0006 – Binary Authorization wired but disabled by default

- **Status:** Accepted
- **Date:** 2026-08-03
- **Deciders:** Platform engineering

Context

Binary Authorization is GKE's admission gate: only container images carrying a valid attestation are allowed to run. It is the control that turns "our CI signs what it builds" into an enforced property rather than a convention, and it is the natural terminus of a supply-chain story.

The obstacle is concrete. Online Boutique's published images are **unsigned** — Google publishes them to a public Artifact Registry without attestations. Enabling enforcement against them means every Pod is rejected at admission and the demo does not start.

Applying our own attestations is not available either: signing images we neither build nor own would be a fiction, and rebuilding them is an explicit [non-goal](#).

Decision

Wire Binary Authorization behind a variable, `enable_binary_authorization`, defaulting to `false`. When true, the cluster is created with `evaluation_mode = "PROJECT_SINGLETON_POLICY_ENFORCE"`.

Document both the reason for the default and the pipeline shape that flips it, in [06-roadmap](#).

Consequences

Positive

- The integration point is visible in code. A reviewer can see exactly where the delivery gate attaches, rather than being told it would go "somewhere in CI".
- Turning it on is a one-variable change once images are signed — no re-architecture.
- The reason for the default is recorded, so it reads as a constraint rather than an omission.

Negative

- **The control is not in force.** Anything with cluster write access can deploy any image from anywhere. This is [gap 3](#), and it is real: a wired-but-disabled control provides no security whatsoever.
- A `dynamic` block that is empty by default is untested in the default path. The enabled path has not been exercised against a signed image here.
- There is a temptation to treat "we have BinAuthz" as a claim. It is not one, and this record exists partly to prevent that.

Alternatives considered

Enable it and accept a broken demo. Rejected: the deliverable must run.

Enable it with a policy allowing everything. The worst option. It produces a cluster that reports Binary Authorization as enabled while admitting any image — a control that exists only to look good on an audit. Rejected on principle: a disabled control is honest, a permissive one is misleading.

Rebuild the upstream images and sign them. Would make enforcement genuine, and would mean forking and maintaining eleven applications to demonstrate one platform control. Rejected as disproportionate, and as modifying the application.

Omit it entirely. Simpler, and rejected because where the delivery gate attaches is exactly the kind of thing this repository is meant to show.

ADR 0007 – Pin upstream manifests to a release tag, not a branch

- **Status:** Accepted
- **Date:** 2026-08-03
- **Deciders:** Platform engineering

Context

`50-deploy.sh` fetches Online Boutique's manifests from the upstream repository over HTTPS and applies them. It originally resolved `master`.

A deployment script that resolves a moving branch has no reproducibility. The same command, run twice a month apart, can deploy different images, different resource requests, even different objects — and nothing in the output says so. Two failures follow:

1. **A demo can break without any local change.** An upstream commit lands between rehearsal and presentation, and the thing you tested is not the thing that runs.
2. **A failure cannot be investigated against the artefact that caused it,** because that artefact is no longer identifiable.

This matters more here than it would elsewhere, because the script's entire purpose is to have the *shape* of a CD job. Non-reproducible delivery is the one property a CD job cannot be allowed to have.

Decision

Resolve manifests from a release tag, defaulting to `v0.10.6`, overridable by environment variable:

```
MANIFEST_VERSION="${MANIFEST_VERSION:-v0.10.6}"  
MANIFEST_URL="https://raw.githubusercontent.com/.../${MANIFEST_VERSION}/release/kubernetes-manifests.yaml"
```

Log the resolved version at apply time, so the deployed release appears in the job output.

Consequences

Positive

- The same command produces the same cluster contents, indefinitely.
- The deployed version is recorded in the log, where an investigation would look for it.
- Evaluating a newer upstream release is one environment variable, and reverting is the same.
- Every count in these documents — 12 Deployments, 11 ServiceAccounts, the resource sums in [05-cost](#) — is now a statement about a specific artefact rather than about whatever upstream currently holds.

Negative

- Upstream fixes, including security fixes, no longer arrive automatically. Someone has to bump the pin, and nothing prompts them to. This trades silent drift for silent staleness, which is better only because it is visible when you look.
- The pin is a second thing to maintain alongside the Terraform provider versions.
- A mistyped tag fails at `kubectl apply` rather than at argument parsing — `set -o errexit` aborts the run, but the error names the URL rather than the mistake.

Alternatives considered

Track `master`. Always current, never reproducible. Rejected for the reasons above.

Vendor the manifests into this repository. Full control and offline deploys, at the cost of owning a copy of eleven applications' Kubernetes configuration and hand-merging upstream changes. Reasonable for a product; disproportionate for a platform demonstration.

Pin to a commit SHA. Strictly more precise than a tag, since tags can in principle be moved. Rejected as marginal: this upstream publishes immutable release tags, and a SHA is meaningless to a human reader, whereas `v0.10.6` can be looked up.

Add a preflight check that the URL returns 200 before touching the cluster. Genuinely worthwhile and not yet done — it would turn a confusing mid-apply failure into a clear one-line error. Deferred, not rejected.

ADR 0008 – An explicit saved-plan gate before apply

- **Status:** Accepted
- **Date:** 2026-08-04
- **Deciders:** Platform engineering

Context

Nothing in this repository gates an infrastructure change. `50-deploy.sh` stands in for the CD stage of a pipeline, but there is no pipeline, so there is no plan-on-PR, no approval step and no second pair of eyes.

Bare `terraform apply` looks like it fills that gap. It prints a plan and waits for `yes`. Two things it does not do:

1. **It re-plans when you confirm.** What executes is generated after you finish reading, not the thing you read. State locking makes divergence unlikely, but unlikely is a different property from guaranteed.
2. **It produces nothing.** The plan exists on a terminal for as long as the scrollback lasts. It cannot be attached to a pull request or read by someone who is not at the keyboard, which is what review actually requires.

Decision

The deploy path uses a saved plan:

```
terraform plan -out=tfplan -var="project_id=$PROJECT_ID"
terraform apply tfplan
```

`apply tfplan` executes that plan or fails. Both [04-runbook](#) and the README quickstart use this form.

Two exemptions, both deliberate:

- **bootstrap/** creates one bucket. The plan is four lines and the ceremony adds nothing.
- **70-destroy.sh** keeps `-auto-approve`. It is run repeatedly, and its safety comes from deleting the Kubernetes namespace before Terraform touches the VPC, not from a prompt.

Consequences

Positive

- The applied change is the reviewed change, or the apply fails.
- The plan file is a reviewable artefact. It is the only thing this repository produces that can be examined *before* a change rather than after.
- It is the same shape as the gate that replaces it. When CI arrives ([06-roadmap](#)), the human approval becomes a pipeline approval and the commands do not change.

- The exemptions are stated, so a reviewer reads them as decisions rather than inconsistency.

Negative

- Two commands where there was one, on every apply.
- Saved plans go stale. If state changes between plan and apply, Terraform refuses the file and the plan has to be regenerated and re-read. Correct behaviour, and still friction.
- Plan files can contain sensitive values. `tfplan` and `*.tfplan` are both gitignored, but a file that exists can be shared carelessly in a way that terminal output cannot.
- **It is a convention, not an enforcement.** Nothing prevents anyone running bare `apply`. Only a pipeline that holds the credentials can make this mandatory.

Alternatives considered

Bare `apply` with the interactive prompt. The status quo. Rejected for the two reasons in the context: it re-plans on confirmation, and it leaves nothing behind.

`-auto-approve` everywhere. Fastest, and no gate at all. Reasonable in a pipeline that has already gated the change upstream; unacceptable as the only path.

Plan-on-PR via HCP Terraform or Atlantis. The real answer, and where this should end up — already recorded as an alternative in [ADR 0004](#). Rejected here as more infrastructure than a single-project deliverable justifies.

Wait for CI and do nothing now. Rejected. The gap is present today, and the cheapest useful gate is one command.